Study Notes: Claude Code Configuration, Hooks & the 10-80-10 Rule

1. TL;DR / Executive Summary

- **Three-Tier Configuration:** Claude Code settings are managed at Global (~/.claude/settings.json), Project (.claude/settings.json), and Local (.claude/settings.local.json) levels for granular control.
- **Event-Driven Automation via Hooks:** Utilizes 33 distinct lifecycle events (session_start, post_tool_use, etc.) structured around "When, Match, and Do" logic.
- **Security & Best Practices:** Recommends automating configuration via skills, enforcing file access restrictions to protect secrets, and managing cross-platform MCP setups.

2. Core Concepts & Definitions

Term / Concept	Definition / Explanation
Global Scope ~/.claude/settings.json	Configuration settings applied universally across all projects on a developer's machine.
Project Scope (.claude/settings.json	Shared team configuration file committed to version control for project-wide consistency.
Local Scope settings.local.json	Personal configuration overrides used by individual developers without impacting team repositories.
Hooks & Events	An automation system driven by 33 distinct lifecycle triggers that execute specific actions when matching conditions occur.
The 10-80-10 Rule	A development framework dividing responsibility into 10% human intent, 80% AI execution, and 10% human verification.
Slash Update Config	A helper skill/command used to automate the generation of settings files instead of manual editing.

3. Structured Body Hierarchy

- **Managing Claude Code Configuration Scopes**
  - **Hierarchical Setup:** Balancing global defaults, shared project rules, and individual local overrides.
  - **Security and Privacy:** Implementing denial rules within configuration files to restrict Claude from accessing sensitive files or secrets.
  - **Memory Customization:** Controlling auto-memory features and configuring custom storage paths to prevent team conflicts.
- **Implementing Hooks and Events**
  - **Event Lifecycle:** Leveraging 33 events (such as session_start or post_tool_use) to automate workflows.
  - **Hook Architecture:** Defined by three components: **When** (the trigger event), **Match** (targeted tools/conditions), and **Do** (the executable action).
- **The 10-80-10 Development Framework**
  - **Phase 1 (First 10%):** Human establishes clear objectives, intent, and architectural guardrails.
  - **Phase 2 (80%):** AI worker handles autonomous execution, tool calls, and drafting.
  - **Phase 3 (Last 10%):** Human verifies outputs, checks compliance, and owns final validation.
- **Best Practices & Cross-Platform Interoperability**
  - **Automation Tools:** Utilizing configuration generation skills to maintain error-free setup files.
  - **Ecosystem Compatibility:** Ensuring persistent workflows and progress can move across providers by correctly configuring Model Context Protocol (MCP) servers.

4. Review & Active Recall (Q&A)

- **Q: What is the difference between project-level (.claude/settings.json) and local-level (.claude/settings.local.json) configurations?**
  - **A:** Project-level settings are shared among team members and tracked in version control, whereas local-level settings are personal overrides unique to an individual developer's environment.
- **Q: What three structural components define a hook in Claude Code?**

- **A: When** (the event trigger), **Match** (the target tool or condition), and **Do** (the action or command to execute).
- **Q: How does the 10-80-10 rule distribute responsibility in AI-assisted development?**
- **A:** 10% human intent definition, 80% autonomous AI execution, and 10% human verification and quality assurance.
- **Q: Why are denial rules useful in Claude Code configuration?**
- **A:** They restrict the agent from reading or interacting with sensitive files (such as API keys or system secrets), protecting data security.